

RaidGuild AI Recipe Helper

Hardware Technical Reference - prototype wiring, components, power, LED states, and pre-assembly checks

1. Project snapshot

This document captures the working hardware configuration for the low-cost countertop voice-first cooking companion. The current prototype uses an ESP32-S3, an INMP441 I2S microphone, a MAX98357A I2S amplifier, a small 4 ohm speaker, two buttons, and a common-cathode RGB LED. The device records a spoken cooking request, uploads a WAV file to the Pinata Agent Chef backend, stores the returned session id, fetches temporary TTS MP3 responses, and plays them through the onboard speaker.

2. Components used

Part	Exact/prototype component	Purpose	Notes
Main controller	ESP32-S3 development board	Wi-Fi, button input, I2S mic input, API calls, MP3 playback	Board has two Micro-USB ports labeled USB and UART. USB port was stable for upload/serial in testing.
Microphone	INMP441 I2S MEMS microphone breakout	Captures voice queries	Use 3.3V only. L/R tied to GND. Header pins must be soldered for reliable readings.
Amplifier	MAX98357A I2S amplifier breakout	Drives speaker from ESP32 I2S audio	VIN powered from 5V/VBUS. SD/EN tied to 3V3 if present. Header pins must be soldered.
Speaker	4 ohm, 3W small speaker	Audio output	Connected only to amp SPK+ and SPK-. Do not connect speaker output to ESP32 GND.
Talk button	Momentary tactile button, 4-leg	Hold-to-record voice query	Uses INPUT_PULLUP. Wire one side to GPIO 18 and the opposite side to GND.
Next button	Momentary tactile button, 4-leg	Advance recipe step	Uses INPUT_PULLUP. Wire one side to GPIO 21 and the opposite side to GND.
Status LED	4-leg common-cathode RGB LED	Device state feedback	Common/longest leg to GND. Each color leg needs its own resistor.
Power	5V USB wall adapter, approx. 1.8A tested	Standalone power	Powered through ESP32 Micro-USB. Wall power test passed.
Prototype wiring	Breadboard + jumper wires	Temporary development wiring	Good for testing; for assembly, move to perfboard or soldered harnesses.

3. Final ESP32-S3 pin map

Subsystem	Component pin	ESP32-S3 pin	Notes
MAX98357A amp	VIN	5V / VBUS	Amp power input.
MAX98357A amp	GND	GND	Shared ground.
MAX98357A amp	SD / EN, if present	3V3	Wakes/enables amp. Leave GAIN disconnected for now.
MAX98357A amp	BCLK / BCK	GPIO 4	I2S bit clock.
MAX98357A amp	LRC / LRCLK / WS	GPIO 5	I2S word select.
MAX98357A amp	DIN / DATA	GPIO 6	I2S audio data into amp.
Speaker	+ / red	MAX98357A SPK+	Speaker connects to amp output, not ESP32.
Speaker	- / black	MAX98357A SPK-	Do not short SPK+ and SPK-.
INMP441 mic	VDD / 3V3	3V3	Do not power INMP441 from 5V.
INMP441 mic	GND	GND	Shared ground.
INMP441 mic	SCK	GPIO 15	I2S mic bit clock.
INMP441 mic	WS	GPIO 16	I2S mic word select.

Subsystem	Component pin	ESP32-S3 pin	Notes
INMP441 mic	SD / DOUT	GPIO 17	I2S mic data output to ESP32 input.
INMP441 mic	L/R	GND	Selects channel used by mic.
Talk button	One side	GPIO 18	INPUT_PULLUP. Press = LOW.
Talk button	Opposite side	GND	For 4-leg button, use opposite sides across the breadboard gap.
Next button	One side	GPIO 21	INPUT_PULLUP. Press = LOW.
Next button	Opposite side	GND	Avoid GPIO 19/20 because they are native USB pins on ESP32-S3.
RGB LED	Red leg	GPIO 8 via resistor	Common cathode LED.
RGB LED	Green leg	GPIO 9 via resistor	Use one resistor per color leg.
RGB LED	Blue leg	GPIO 10 via resistor	Use one resistor per color leg.
RGB LED	Common/longest leg	GND	Common cathode confirmed by test sketch.

Pins to avoid

Avoid GPIO 19 and GPIO 20 while using the ESP32-S3 native USB port. These are commonly tied to USB D- and USB D+. Using them for buttons caused a likely USB-disconnect/debugging risk during development.

4. Wiring notes by subsystem

Speaker / amplifier

The MAX98357A receives digital I2S audio from the ESP32-S3 and drives the 4 ohm speaker. The ESP32 does not drive the speaker directly. The amp VIN is powered from the ESP32 5V/VBUS pin, while the SD/EN pin, if present, is tied to 3V3. The speaker wires connect only to SPK+ and SPK- on the amp. Header pins on the amp must be soldered; unsoldered headers caused intermittent screeching and no playback during early testing.

Microphone

The INMP441 is a digital I2S MEMS microphone. It must be powered from 3V3, not 5V. The L/R pin is tied to GND. The mic breakout should straddle the breadboard center gap if it has a 2x3 pin layout. Header pins must be soldered; before soldering, only wire movement changed the mic readings.

Buttons

Both buttons use the ESP32 internal pull-up resistor. The button does not need a power wire. One side goes to a GPIO pin and the opposite side goes to GND. For 4-leg tactile buttons, legs on the same side are often internally connected. If the button reads as permanently pressed or never pressed, rotate it 90 degrees or move the wires to opposite sides across the breadboard center gap.

RGB LED

The RGB LED is a 4-leg common-cathode LED. The common/longest leg goes to GND. The red, green, and blue legs each connect through their own resistor to GPIO 8, GPIO 9, and GPIO 10 respectively. Common cathode means HIGH turns a color on and LOW turns it off. Colors can be remapped in code if the physical color order differs.

5. Power model

Power path	Recommended value	Notes
Wall adapter	5V, 1A minimum; 5V, approx. 1.8A tested	A Google USB wall adapter labeled around 5V/1.8A is appropriate. Amperage is available capacity; the ESP32 only draws what it needs.
ESP32 input	Micro-USB port	Use the same Micro-USB port that has been stable for upload/testing, typically the USB-labeled port.
Amp power	ESP32 5V/VBUS -> MAX98357A VIN	Keeps amp on 5V for better speaker output.

Power path	Recommended value	Notes
Mic power	ESP32 3V3 -> INMP441 VDD	Do not connect mic VDD to 5V.
Shared ground	All GNDs tied together	ESP32, amp, mic, buttons, and LED common cathode must share ground.

6. LED status behavior

Color	Meaning	Prototype behavior
Off	Idle or no power	Not used as normal ready state; off still means no visible activity.
White	Bootng or initializing	Set at firmware startup.
Blue	Ready and operating normally	Solid blue when ready; pulsing blue while playing response.
Green	Task completed successfully / active success	Brief flash after a successful API response and playback.
Yellow	Busy, processing, waiting for input	Connecting Wi-Fi, recording, uploading/thinking.
Red	Error, fault, or blocked state	Blinking red on failure.
Purple	Setup, pairing, or special mode	Available in code for future setup mode; not currently active.

7. Current firmware behavior

Action	Firmware behavior	Backend interaction
Hold Talk	Records mic audio while button is held, caps recording length, converts PCM to WAV	Uploads multipart form-data to POST /app/query-audio with field audio=recording.wav and optional sessionId.
Voice response	Parses JSON, stores session.id, extracts audio.url	Backend returns transcript, intent, answerText, audio.url, and session.
TTS playback	Fetches temporary MP3 URL, handles chunked response, dechunks, decodes MP3 from memory, plays through MAX98357A	audio.url is temporary and should be fetched promptly.
Press Next	Sends JSON next_step using stored sessionId	POST /app/query-audio with {inputMode:'next_step', sessionId:'...'}; then plays returned audio.url.

Important audio implementation notes

The uploaded WAV uses a header sample rate that was tuned during testing to improve transcription quality. Earlier local playback tests revealed a speed mismatch; adjusting the WAV header sample rate made the backend transcription much more accurate. MP3 response playback required manual handling of HTTP chunked transfer encoding before decoding the MP3 from memory.

8. Pre-assembly checklist

Category	Checklist item
Firmware	Boots, connects to Wi-Fi, records Talk audio, uploads WAV, stores session.id, plays returned MP3, and sends Next successfully.
Wall power	Runs full Talk/Next flow from 5V wall adapter without laptop; no brownouts, resets, or Wi-Fi drops.
Wiring	Gently wiggle amp, mic, buttons, LED, and power wiring while running. No crackle, LED flicker, false presses, or resets.
Mic placement	Mic port faces user/front-up direction. Do not block the mic hole with plastic, wires, tape, or fingers.
Speaker placement	Speaker grille/opening is clear. Avoid pointing speaker directly at mic to reduce feedback and re-recorded audio.
Buttons	Talk and Next buttons are reachable without moving the box. 4-leg buttons are oriented correctly.
LED	LED is visible from normal countertop angle. Resistors are secure and not shorting adjacent rows.
Enclosure	USB cable exits cleanly. No pressure on ESP32 reset/boot buttons. Enough clearance for jumper wires.

Category	Checklist item
Permanent build	Move from breadboard to perfboard/soldered harness/JST-style connectors before daily countertop use.

9. Troubleshooting notes from the prototype

Symptom	Likely cause	Fix
Amp/speaker only works when held	Unsoldered amp header pins	Solder header pins to MAX98357A board.
Mic levels only change when wires move	Unsoldered mic headers or poor breadboard contact	Solder mic header pins; reseal module across breadboard gap.
ESP32 disconnects when Next button wired	GPIO 19/20 native USB conflict risk	Use GPIO 21 for Next and avoid GPIO 19/20.
Button stuck pressed	4-leg button wired on same internally connected side	Wire GPIO and GND to opposite sides/diagonal across breadboard gap.
MP3 URL works in browser but fails on ESP32	HTTPS/chunked transfer handling	Download to memory, strip chunk framing, then decode MP3 from memory.
Transcription poor despite web simulator working	WAV sample-rate/header mismatch or overprocessed audio	Use cleaner processing and tuned WAV header sample rate.
Audio sounds thin	Small speaker with no enclosure	Expected on bench. Enclosure cavity and grille can improve perceived fullness.

10. Assembly recommendation

For a reliable countertop prototype, avoid leaving the full build on a solderless breadboard. At minimum, solder the amp and mic headers, secure the speaker wires, and strain-relieve the power cable. A better next build is a small perfboard or soldered harness where the ESP32, mic, amp, buttons, LED resistors, and shared ground connections cannot wiggle loose inside the enclosure.